A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)

GSIRE An ISO 9001:2015 Certified Institute

STUDENT GRADE CALCULATOR & REPORT CARD

A seminar report submitted

In partial fulfilment of the requirements for the Degree of

Bachelor of Technology

In

Information Technology

Semester – V

By

Patanwadia Diya (12402080601038)

Faculty Guide

Nayan Mali

Assistant Professor

Academic Year 2026-27 (Odd)

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar

Vitthal Udyognagar - 388121
Anand (Gujarat), India

CERTIFICATE

This is to certify that seminar work embodied in this report entitled, "**STUDENT GRADE CALCULATOR & REPORT CARD**" was carried out by **Patanwadia Diya(12402080601038)** at A D Patel Institute of Technology for partial fulfilment of B.Tech. degree to be awarded by the Charutar Vidya Mandal (CVM) University. This seminar work has been carried out under my supervision and is to the satisfaction of department.

Date:

Place:ADIT

Guide	**Head of Department**
Nayan Mali	Dr. Priyang Bhatt
Assistant Professor	Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my seminar guide, Prof. Nayan Mali for their valuable guidance, encouragement, and support throughout the preparation of this report. I am also thankful to Dr. Priyang Bhatt, Head of Department, for providing the necessary facilities and academic environment. Finally, I extend my appreciation to my family and friends for their constant motivation.

Patanwadia Diya

(12402080601038)

INDEX

Chapter	Title	Page No.
-	Certificate	ii
-	Acknowledgement	iii
-	Abstract	2
1	Introduction	3
2	Tools And Technologies	4
3	System Design and Working	5
4	Implementation	7
5	Testing and Results	9
6	Limitations and Future Scope	10
7	Conclusion and References	11

ABSTRACT

The Student Grade Calculator & Report Card is a Python-based mini-project developed to automate the calculation of students' marks, percentages, grades, and final results. The main purpose of this project is to reduce manual calculation errors and make the grading process faster, simpler, and more accurate.

The program accepts the student's name, subject names, and marks for 3 to 5 subjects, each carrying a maximum of 100 marks. It calculates the total marks and percentage, assigns a grade using conditional statements, and displays a properly formatted report card. If a student scores below 35 marks in any subject, the final result is Fail, irrespective of the overall percentage. The program also includes input validation and exception handling to prevent invalid entries and unexpected errors.

The project uses fundamental Python concepts such as functions, loops, if-elif-else statements, try-except blocks, and formatted output. It provides practical experience in modular programming and problem-solving while demonstrating how simple programming techniques can automate an academic task. The project can be further enhanced by adding a graphical user interface, database connectivity, and the facility to store and process results for multiple students.

CHAPTER 1: INTRODUCTION

The Student Grade Calculator & Report Card is a Python-based mini-project designed to automate the process of calculating students' marks, percentages, grades, and pass/fail results. Manual calculation of marks can be time-consuming and may lead to arithmetic errors. This project provides a simple and consistent way to generate a report card by entering the student's name, subject names, and marks.

The program supports 3 to 5 subjects, with each subject carrying a maximum of 100 marks. It calculates the total marks and percentage, assigns a grade using conditional statements, and displays the result in a properly formatted report card. If a student scores below 35 marks in any subject, the final result is Fail, regardless of the overall percentage.

The project uses basic Python concepts such as functions, loops, conditional statements, exception handling, and formatted output. It runs through a console interface and does not require any external libraries.

1.1 Problem Statement

Manual preparation of report cards requires repeated calculations and checking of grade cut-offs. This can result in errors and inconsistent grading. The project aims to solve this problem by automating calculations and applying the same grading rules to every student.

1.2 Objectives

- To accept a student's name and marks for 3 to 5 subjects.
- To validate inputs and reject invalid marks or subject counts.
- To calculate total marks, maximum marks, and percentage.
- To assign grades using if-elif-else statements.
- To display a clear report card with the final result.
- To use modular programming by dividing the program into functions.

CHAPTER 2: TOOLS AND TECHNOLOGIES

The project uses Python and its built-in features, making it suitable for beginners and easy to execute on different operating systems.

Component	Description
Programming language	Python 3.8 or later
Code editor	VS Code, IDLE, or PyCharm
Libraries	No external libraries required
Operating system	Windows, Linux, or macOS
Interface	Console-based input and output
Error handling	try-except statements

Table 2.1: Tools and Technologies Used

2.1 Python Concepts Used

The program uses conditional statements to assign grades, loops to repeat inputs and process subjects, functions to separate tasks, exception handling to manage invalid input, and f-strings to display neatly aligned results.

Concept	Application
if-elif-else	Determines the final grade
for loop	Processes subject marks
while loop	Repeats input until it is valid
Functions	Divides the program into manageable tasks
try-except	Prevents crashes caused by non-numeric input
F-strings	Formats the report card output

Table 2.2: Python Concepts and Their Uses

2.3 Advantages of Using Python

Python makes the project easy to develop and understand. Its built-in functions reduce the amount of code required, while its conditional statements and loops simplify grading and validation. The use of functions also improves readability and makes individual sections easier to test.

CHAPTER 3: SYSTEM DESIGN AND WORKING

3.1 System Overview

The Student Grade Calculator follows a sequential process in which the input is collected, validated, processed, and displayed as a report card.

The system consists of four main stages:

1. Input: Collect the student's name, subject count, subject names, and marks.

2. Validation: Ensure that the subject count and marks are within the permitted limits.

3. Processing: Calculate totals and percentage, then determine the grade and result.

4. Output: Display the final report card and ask whether another student should be processed.

3.2 System Flowchart

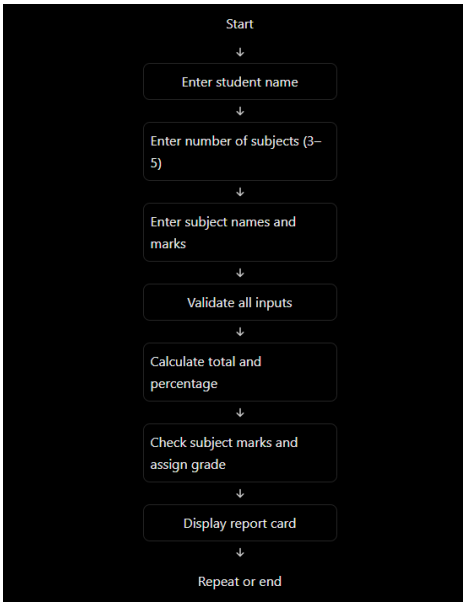


Figure 3.1: Flowchart of the Student Grade Calculator

3.3 Mathematical Formulas

The program uses the following formulas to calculate the student's performance.

Total Marks

$$\text{Total Marks}=\sum_{i=1}^{n} M_i$$

Here, M_i represents the marks obtained in each subject.

Maximum Marks

$$\text{Maximum Marks}=n\times100$$

Percentage

$$\text{Percentage}= \frac{\text{Total Marks}}{\text{Maximum Marks}}\times100$$

Here, n represents the number of subjects entered by the user.

Example Calculation :

Suppose a student obtains 75, 80, and 65 marks in three subjects.

- Total marks = 75 + 80 + 65 = 220
- Maximum marks = 3 × 100 = 300
- Percentage = (220 ÷ 300) × 100 = 73.33%

Since all subject marks are at least 35 and the percentage is between 60% and 75%, the student receives Grade B.

3.4 Grading Rules

The system uses percentage-based grading along with a subject-wise passing condition.

Condition	**Grade/Result**
Percentage ≥ 75% and all subjects passed	A
Percentage ≥ 60% and below 75%, all subjects passed	B
Percentage ≥ 40% and below 60%, all subjects passed	C
Percentage below 40%	Fail
Any subject below 35 marks	Fail regardless of percentage

Table 3.1: Grading Criteria

The subject-wise passing condition is checked before assigning a percentage-based grade. This prevents a student with a high overall percentage but a failed subject from incorrectly receiving a passing grade.

CHAPTER 4: IMPLEMENTATION

4.1 Program Structure

The program is divided into multiple functions, with each function responsible for a specific task. This makes the code easier to understand and reduces repetition.

Function	Responsibility
get_valid_number()	Accepts a number within a specified range
get_student_details()	Collects the student's information and subject marks
calculate_results()	Calculates totals and percentage
assign_grade()	Determines the grade and pass/fail result
display_report_card()	Prints the formatted report card
main()	Controls the program's overall execution

Table 4.1: Functions Used in the Program

4.2 Grading Logic in Python

The assign_grade() function applies the grading rules using conditional statements. The first condition checks whether the student has passed all subjects. If not, the function immediately returns Fail.

```
def assign_grade(percentage, passed_all_subjects):
    if not passed_all_subjects:
        return "Fail"

    if percentage >= 75:
        return "A"
    elif percentage >= 60:
        return "B"
    elif percentage >= 40:
        return "C"
    else:
        return "Fail"
```

This code is easy to maintain because the grading conditions are arranged from the highest percentage threshold to the lowest. If the grading system changes in the future, the thresholds can be updated in this function.

4.3 Input Validation and Error Handling

Input validation is an important part of the project. The program must reject invalid subject counts and marks outside the range of 0 to 100. It also handles non-numeric input using a try-except block.

User Input	Expected Program Response
Subject count: 2	Requests a number between 3 and 5
Subject count: 6	Requests a number between 3 and 5
Marks: 105	Requests marks between 0 and 100
Marks: -5	Requests marks between 0 and 100
Marks: abc	Displays an invalid-input message

Table 4.2: Examples of Input Validation

This feature makes the program more reliable and user-friendly. Instead of terminating when the user enters an incorrect value, it requests the input again.

4.4 Report Card Generation

After completing the calculations, the program displays the student's information in an organised format. The report card contains the student name, subject-wise marks, total marks, maximum marks, percentage, final grade, and result.

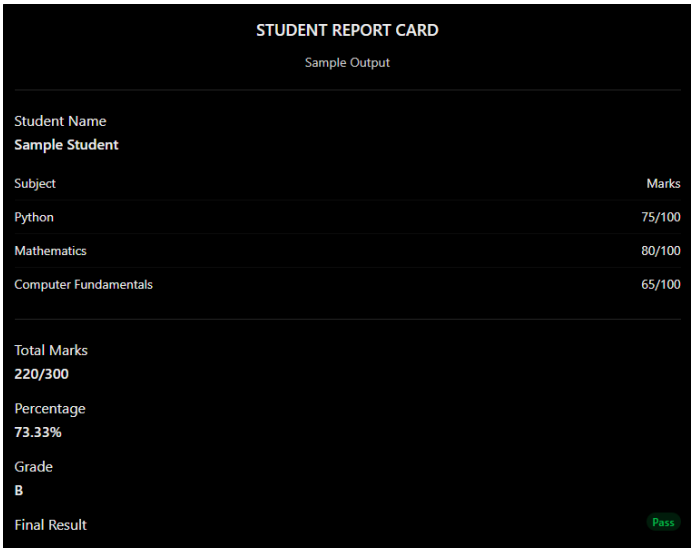


Figure 4.1: Sample Report Card Layout

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Objectives

Testing is performed to verify that the program calculates results correctly and follows the required grading rules. It also checks whether the program rejects invalid input and handles the subject-wise passing condition correctly.

The main objectives of testing are to verify percentage calculations, check grade boundaries, confirm that marks below 35 result in failure, and ensure that invalid inputs do not interrupt execution.

5.2 Test Cases

Subject Marks	Total	Percentage	Grade
75, 75, 75	225/300	75.00%	A
70, 65, 60, 55	250/400	62.50%	B
60, 60, 60	180/300	60.00%	B
50, 45, 40	135/300	45.00%	C
40, 40, 40	120/300	40.00%	C
35, 35, 35	105/300	35.00%	Fail
78, 30, 66, 70	244/400	61.00%	Fail

Table 5.1: Test Cases and Results

5.3 Analysis of Results

The test cases confirm that the program correctly calculates percentages and assigns grades according to the defined rules. A student scoring below 35 in any subject receives a Fail result, even with an overall percentage of 61%. Input validation also handles invalid subject counts, out-of-range marks, and non-numeric entries effectively.

5.4 Expected Outcomes

The expected outcome is a working console-based application that produces a report card with the correct total, percentage, grade, and result. It should also apply consistent grading criteria and allow users to process another student after the previous report card has been displayed.

CHAPTER 6: LIMITATIONS AND FUTURE SCOPE

6.1 Limitations

Although the project successfully performs basic grade calculations, it has some limitations.

- It processes only one student at a time.
- It supports a maximum of five subjects.
- Each subject is restricted to 100 marks.
- Results are displayed in the console and are not permanently saved.
- The grading scale is fixed in the program.
- It does not include a graphical interface or database.

These limitations do not affect the main purpose of the project but provide opportunities for future improvement.

6.2 Future Enhancements

The system can be expanded with additional features to make it more useful in educational environments.

Enhancement	Expected Benefit
File handling	Save report cards for later use
Tkinter GUI	Provide a graphical interface with buttons and input fields
Student class using OOP	Organise student data and related operations
Multiple-student processing	Calculate and compare results for an entire class
Database connectivity	Store and retrieve student records
Custom grading scales	Support different pass marks and grade thresholds

Table 6.1: Proposed Future Enhancements

Adding file handling would prevent results from being lost when the program closes. A graphical interface could make the system easier for non-technical users, while database integration could support long-term storage and retrieval of student records.

Processing an entire class would also allow the program to compare student performance and rank students by percentage. These improvements could transform the current beginner-level project into a more comprehensive student result management system.

CHAPTER 7: CONCLUSION AND REFERENCES

7.1 Conclusion

The Student Grade Calculator & Report Card project demonstrates the practical application of Python programming in automating a common academic task. It calculates total marks and percentages, assigns grades according to predefined conditions, and generates a structured report card. By reducing the need for repeated manual calculations, the system helps minimise arithmetic errors and maintains consistency in the grading process.

The project also provides valuable experience in using functions, loops, conditional statements, lists, exception handling, and formatted output. Input validation improves reliability by ensuring that incorrect values are not passed into the calculation process. Testing further demonstrates that the program follows the defined grading rules, including the important condition that a student who scores below 35 in any subject receives a Fail result.

Although the current version is limited to console-based interaction and does not store records permanently, it provides a useful foundation for future development. Features such as a graphical user interface, database connectivity, file handling, and multi-student processing could make the system more versatile and practical.

Overall, the project achieves its primary objective of simplifying grade calculation and report card generation while strengthening the understanding of fundamental Python programming concepts.

7.2 References

1. Python Software Foundation. The Python Tutorial. Available at: https://docs.python.org/3/tutorial/

2. Python Software Foundation. More Control Flow Tools. Available at: https://docs.python.org/3/tutorial/controlflow.html

3. Python Software Foundation. Errors and Exceptions. Available at: https://docs.python.org/3/tutorial/errors.html

4. Python Software Foundation. Input and Output. Available at: https://docs.python.org/3/tutorial/inputoutput.html

5. Python Software Foundation. Built-in Functions. Available at: https://docs.python.org/3/library/functions.html